GRIT Interview Hub — submission write-up

Problem. A student completes the required skills, books one of a limited set of live interview slots, is evaluated, and either earns a badge or gets a dated retake window. Operations must see who is eligible, booked and unbooked, publish results, handle reassignment and malpractice, and never lose the history behind a badge. This is a working Next.js 16 + PostgreSQL application, not a mockup.

1. States and what moves them

Journey: `NOT_ELIGIBLE → ELIGIBLE → (booking) CONFIRMED → SCHEDULED → IN_PROGRESS → COMPLETED → RESULT_PENDING → RESULT_PUBLISHED → badge`.

Stage	States	Moves on when (owner)
Eligibility	`NOT_ELIGIBLE`, `ELIGIBLE`, `ELIGIBILITY_REVOKED`	Every required skill completes → `ELIGIBLE` (system). Operations can revoke/restore; revoked students cannot book
Booking and slot	`CONFIRMED`, `CANCELLED`, `RESCHEDULED`, `NO_SHOW`, `COMPLETED`; slot `OPEN`, `BOOKED`, `CANCELLED`, `BLOCKED`	Student books atomically; cancels/reschedules outside 4 hours; operations records no-show; completing closes it
Interview	`SCHEDULED`, `IN_PROGRESS`, `COMPLETED`, `RESULT_PENDING`, `RESULT_PUBLISHED`	Interviewer starts and completes, then submits the evaluation; operations publishes
Result	`PASS`, `FAIL`, `RETAKE_LATER`	Set on publication and terminal; never edited
Badge	`NOT_EARNED`, `PENDING_CLEARANCE`, `AWARDED`, `SUSPENDED`, `NULLIFIED`	A pass awards it; a confirmed integrity case suspends or nullifies while the result record stays
Retake	`PENDING`, `APPROVED`, `REJECTED`, `SCHEDULED`, `COMPLETED`	Requested after `FAIL/RETAKE_LATER`; operations decides; approval unlocks booking
Integrity	`REPORTED`, `UNDER_REVIEW`, `CONFIRMED`, `REJECTED`, `CLOSED`	Operations investigates and closes the case with a badge action

Every transition is validated by one state machine (`src/lib/state-machine.ts`) plus conditional database updates, so a stale tab cannot force an illegal move.

2. Edge cases and assumptions

Edge case	Behaviour
Two students race for one slot	Row locks + conditional update; exactly one wins and the other sees “that slot was just taken” with refreshed availability
Student already holds a confirmed booking	Rejected by a service check and a partial unique index
Double-click / retried request	A unique idempotency key returns the original booking
Slot inside the 60-minute lead time	Not offered and rejected server-side
Cancel/reschedule inside 4 hours	Blocked with the policy explained; outside it, the old booking becomes `RESCHEDULED` and stays in history
Interviewer blocks availability with confirmed bookings	Interviews move to the operations reassignment queue; students are notified
No-show	`NO_SHOW` is recorded; no result is invented
Evaluation edited, or result published twice	State machine rejects both; submitted evaluations are immutable
Fail/retake-later	`retakeEligibleFrom` (14/21-day waits), early review request, attempt caps (3/2)
Malpractice confirmed after a badge	Badge suspended or nullified with a reason; result history preserved and the student notified
Eligibility revoked after booking	Booking stays for operations to reassign; the student sees the hold

Assumptions: one timezone (`Asia/Kolkata`), 45-minute slots, one active interview type per journey, operations is the only publisher, in-app notifications only, no payments. **Open questions:** should interviewers see a student’s prior feedback, and should retakes use the same interviewer cohort?

3. Data model and double-booking prevention

Table	Key fields	Purpose
`User`, `StudentProfile`, `InterviewerProfile`	`email`, `role`; `studentCode`, `eligibilityStatus`, `cohort`; `maxPerDay`	Identity and profiles
`Skill`, `StudentSkill`, `Badge`, `BadgeAward`	`code`, `isRequired`; `status`; `studentId+badgeId`, `awardedAt`	Eligibility and badge clearance
`InterviewerAvailability`, `InterviewSlot`	window/date, `slotMinutes`; `startsAt`, `endsAt`, `status`	Availability and bookable slots

Table	Key fields	Purpose
`InterviewBooking`	`studentId`, `slotId`, `status`, `idempotencyKey`, `rescheduledFromId`	Never-deleted booking history
`Interview`, `InterviewEvaluation`, `InterviewResult`	`status`, `outcome`, `retakeEligibleFrom`; three scores, recommendation; `publishedAt`	Session, evaluation, result
`RetakeRequest`, `MalpracticeCase`	`status`, `eligibleFrom`; `status`, `badgeAction`, `allegation`, `resolution`	Exception workflows
`Notification`, `AuditLog`	`type`, `body`, `link`; actor, action, before/after	Messages and explainability

Double-booking prevention. The booking transaction locks the student and slot rows (`SELECT … FOR UPDATE`), re-checks eligibility and slot state inside the lock, then flips the slot with `updateMany({ where: { id, status: "OPEN" } })`; the winner must affect exactly one row. Two partial unique indexes are the database backstop — **one `CONFIRMED` booking per student** and **one per slot**. A PostgreSQL integration test races two students with `Promise.allSettled` and asserts one success, one conflict, and one booking.

4. Three most important student messages (exact copy)

Eligibility unlocked, when the last required skill completes:

> Congratulations! You have completed the required skill milestones and are now eligible for your GRIT interview. Choose an available slot to begin your badge clearance journey.

Booking confirmed, immediately after the slot is reserved:

> Your GRIT interview has been successfully booked. Your interview details are available in your dashboard. Please review your scheduled date and time and be ready before the interview begins.

Result published, on a pass; the badge is awarded at the same time:

> Congratulations! You have successfully cleared your GRIT interview. Your badge has been awarded and is now available in your dashboard.

Result variants (`FAIL` / `RETAKE_LATER`) replace the last sentence with the retake instructions. All copy lives in `src/lib/notifications.ts` and is asserted by tests.

5. The prototype, including before/after

- **Student:** `/student` (status, next action, clearance path, result, badge, retakes), `/student/book` (real availability and confirmation dialog), `/student/interviews`, `/student/badge`.
- **Interviewer:** `/interviewer` (today), `/interviewer/availability` (set/block windows), `/interviewer/interviews/[id]` (schedule, evaluation with three scores, strengths, improvements, recommendation).
- **Operations:** `/admin` queues, `/admin/students` with booked/unbooked filtering and the “eligible but unbooked” queue, results, retakes, malpractice, audit and analytics.
- **Before/after:** `/result-impact` renders the same dashboard from real seeded records — Kabir Shah before publication, Meera Krishnan after a pass and badge, Rohan Verma after a fail with a dated retake window. Linked from the landing and login screens.

Demo sign-in is by persona (Student / Interviewer / Interview operations), no password. Live URL is added after Vercel and hosted Postgres are linked; this document does not invent one.